CORE JAVA SERIES — DAY 13

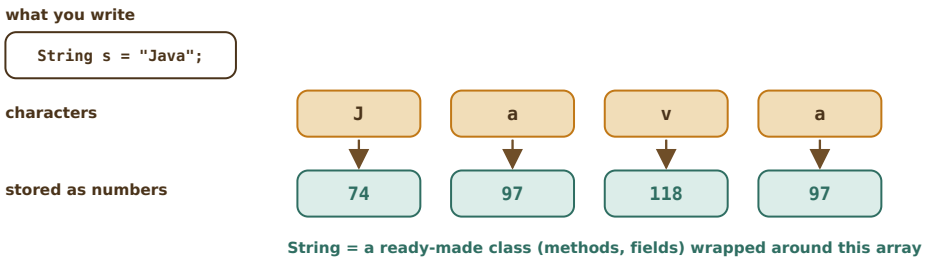
Strings in Depth

What a String really is, why it is immutable, the String pool and its golden rule, the fields inside the String class and the memory tricks Java plays with them — then the String methods grouped by purpose, and the mutable StringBuilder and StringBuffer.

What a String Is

A **character** is any symbol a human can read: letters, digits, punctuation, and the letters of any language. In Java it is held by the primitive type `char`. A **String** is simply a **sequence of characters**. In programming a sequence is stored in an array, so a String is an abstraction on top of a character array.

Inside memory a character is never stored as a picture; its numeric **Unicode value** is stored (for example `'a'` is 97), and the array holds those numbers.



A String wraps an array of character codes.

Why not just use `char[]` everywhere? Because a raw array has none of the everyday operations: comparing, concatenating, taking a substring, searching — you would have to code each algorithm by hand. Strings are among the most used data types (names, passwords, emails, URLs, hashes), so Java wraps the array in a class that carries all those operations. A String is therefore a **non-primitive type**, i.e. a class, living in `java.lang` (auto-imported, nothing to import). It plays the same role for `char[]` that `Integer` plays for `int`.

Immutability

The most important property of String: it is an **immutable class**. Once a String object exists, its content can never change. Why did Java choose that? Strings carry passwords, URLs and hash keys. If a String could change underneath you, a password could silently change and the keys of a `HashMap` or `HashSet` could change, collapsing the whole structure. So any “change” creates a new object and leaves the original alone.

```
String s1 = "Hello";
s1.concat(" World");            // builds a NEW string "Hello World" ... and throws it away
System.out.println(s1);         // Hello   <- the original never changed

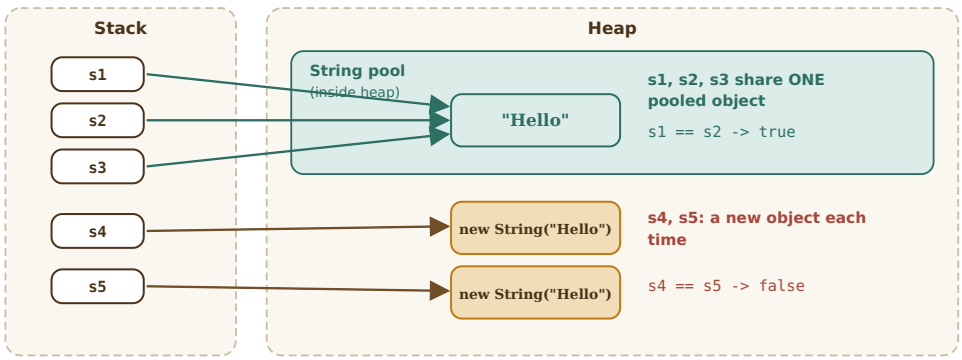
s1 = s1.concat(" World");       // only now does s1 point to the new object
System.out.println(s1);         // Hello World
```

Two Ways to Create a String

Java lets you create a String in two ways, and the difference decides *where in memory* the object lives:

Way	Syntax	Lives in
Literal	`String s = "Hello";`	The **String pool**: reused if the same text already exists
new operator	`String s = new String("Hello");`	Normal **heap**: a brand-new object every time

A program uses millions of Strings, so creating a fresh object for every one would flood memory. For literals the JVM keeps a special area called the **String pool** (since modern Java it is part of the heap; it used to live in a separate area called PermGen). When you write `String s = "Hello"` the JVM first checks whether `"Hello"` already exists in the pool. If yes, `s` just points to it; if not, one object is created there. The `new` operator skips this and always builds a separate object in the normal heap.



Literals share one pooled object; each new String(...) builds its own heap object.

== versus equals()

`==` compares *references* (do they point to the same object?). `Object.equals()` also does that by default, but **String overrides** `equals()` so that it compares the *text*. That gives the classic results:

```
String s1 = "Hello", s2 = "Hello";
System.out.println(s1 == s2);                // true   same pooled object

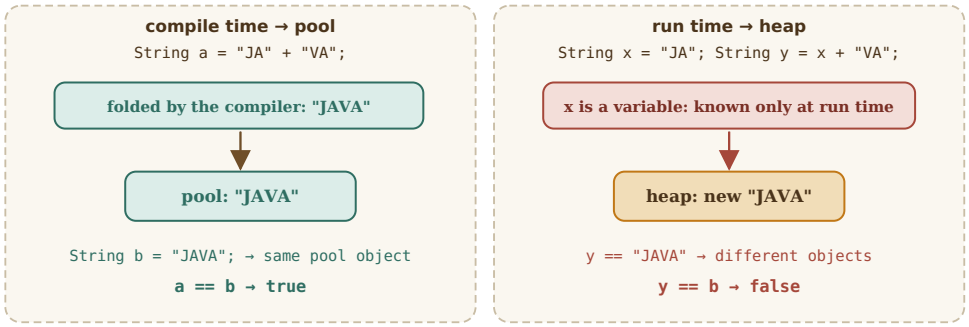
String s3 = new String("Adnan");
String s4 = new String("Adnan");
System.out.println(s3 == s4);                // false  two heap objects
System.out.println(s3.equals(s4));           // true   same text
```

> To compare Strings by content, always use `equals()` (or `equalsIgnoreCase()`), never `==`.

The Golden Rule of the String Pool

Almost every tricky Strings question reduces to one rule:

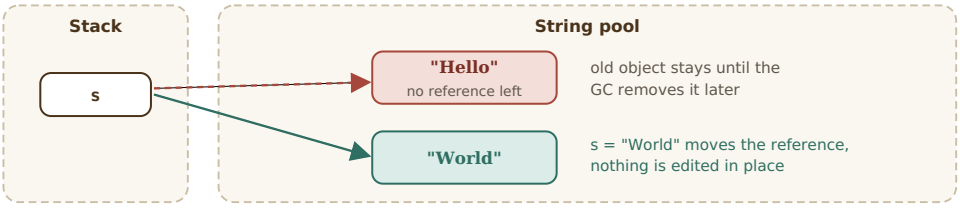
> **Only compile-time constants go to the String pool. Anything created at run time — through a variable, a calculation or the `new` operator — goes to the normal heap.**



Two literals joined with + are folded at compile time; a variable in the expression forces run time.

Work through each case with the rule:

Code	Where the result lives	Comparison
`String a = "JA" + "VA";` `String b = "JAVA";`	Both literals are joined by the compiler → pool; `b` reuses it	`a == b` → **true**
`String x = "JA";` `String y = x + "VA";`	`x` is a variable, so the join happens at run time → heap. (`"VA"` itself, being a literal, still lands in the pool.)	`y == "JAVA"` → **false**
`String p = "Java";` `String q = p;`	Plain assignment copies the reference → both point to the same pooled object	`p == q` → **true**
`String s = "Hello";` `s = "World";`	"World" is a new pooled object; `s` now points to it. "Hello" stays until nobody references it and garbage collection removes it.	`s` prints **World**
`String n = new String("Hello");` `String m = "Hello";`	`new` → heap object; but the literal passed to the constructor is also placed in the pool, and `m` points to that pool copy	`n == m` → **false**



Reassigning a String variable moves the reference; the old object is never edited.

```
String a = "JA" + "VA";           String b = "JAVA";
System.out.println(a == b);                  // true   (compile-time constant)

String x = "JA";                String y = x + "VA";
System.out.println(y == b);                  // false  (built at run time, in the heap)

String s9  = new String("Hello"); String s10 = "Hello";
System.out.println(s9 == s10);               // false  (heap vs pool)
```

Active Recall — Part 1

Q1. What is a String, and why not just use char[]?
A. A sequence of characters wrapped in a class; char[] lacks comparing, concatenating, substring and other operations.

Q2. Why did Java make String immutable?
A. Strings hold passwords, URLs and hash keys; if they could change, hash-based collections and security would break.

Q3. Where does each creation style put the object?
A. Literal → String pool (reused). new → normal heap (always a new object).

Q4. State the golden rule.
A. Only compile-time constants go to the pool; run-time results (variables, calculations, new) go to the heap.

Q5. What do "JA"+"VA"=="JAVA" and (x+"VA")=="JAVA" give, with x = "JA"?
A. true (folded at compile time) and false (built at run time).

The Cost of Immutability

Immutability has a price: every modification builds a new object. Consider building a string in a loop:

```
String s = "";
for (int i = 0; i < 5; i++) {
    s += i;                         // s = s + i  -> a NEW String every iteration
    System.out.println(s);          // 0   01   012   0123   01234
}
```



5 loop iterations of s += i → 6 separate objects
only the last one is still referenced by s; the other five are garbage

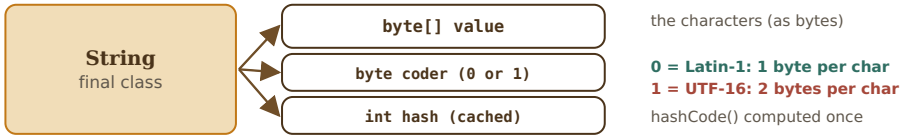
Five iterations leave five abandoned objects waiting for garbage collection.

Each pass creates a longer String in the heap and leaves the previous one unreferenced. For a loop of thousands of iterations this wastes a lot of memory. Java's answer is a pair of **mutable** classes, `StringBuilder` and `StringBuffer`, covered below.

Inside the String Class

Conceptually the class looks like this:

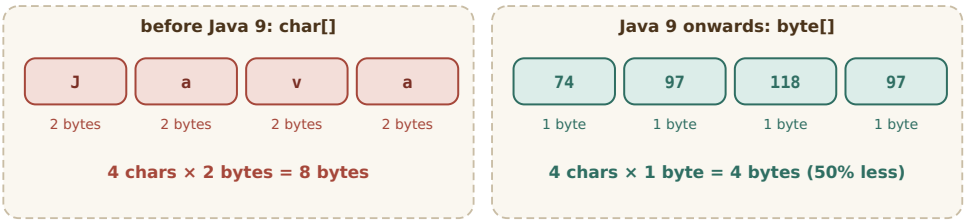
```
public final class String {                // final: nobody can extend it
    private final byte[] value;                 // the characters (as bytes)
    private final byte   coder;                 // 0 = LATIN1, 1 = UTF16
    private int          hash;                  // cached hashCode()
    ...
}
```



The three important fields of String.

value: byte[] instead of char[]

Up to Java 8 the text was kept in a `char[]`, and a Java `char` takes **2 bytes** (it must be able to represent any Unicode symbol). So `"Java"` cost 4 × 2 = 8 bytes. But most Strings contain only ordinary keyboard characters — English letters, digits, symbols — and these fit in **one byte** (values 0–255, the Latin-1 range). From Java 9 the field is a `byte[]` and `"Java"` is stored as 74, 97, 118, 97 in just 4 bytes: a 50% saving.

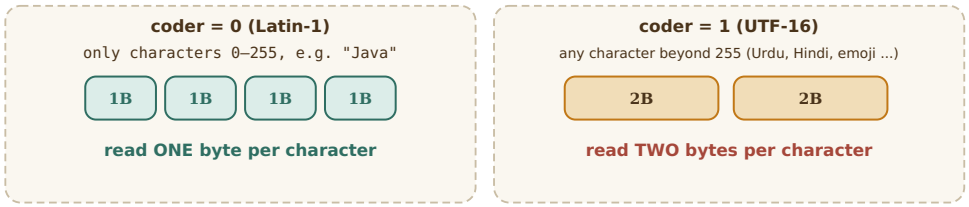


Same text, half the memory once the array became byte[].

coder: how to read the bytes

What if the String contains a character outside the one-byte range, such as an Urdu or Hindi letter or an emoji? Those need two bytes. The `coder` field tells the String class how to read `value`:

coder	Format	Read
0	Latin-1 (every character fits in one byte)	one byte at a time = one character
1	UTF-16 (at least one character needs two bytes)	two bytes at a time = one character



coder decides whether the byte array is read one byte or two bytes per character.

hash: a cache that immutability makes safe

`hashCode()` involves arithmetic with prime numbers over every character — not free. The String class therefore computes it once and stores it in the `hash` field; later calls return the stored number. This is only legal **because the String is immutable**: the content, and so the hash, can never change. Hash collections such as `HashMap` and `HashSet` depend on this.

> Three optimisations make Strings cheap: the **pool** (fewer objects), **byte[] with coder** (about half the memory for ordinary text) and **hash caching** (no repeated hash work).

Active Recall — Part 2

Q1. What is the cost of immutability, and Java's fix?
A. Each change creates a new object and wastes memory; use mutable StringBuilder / StringBuffer.

Q2. Name the three fields of String.
A. byte[] value, byte coder, int hash.

Q3. Why did Java 9 switch from char[] to byte[]?
A. char takes 2 bytes; most text fits in 1 byte (Latin-1), so memory is roughly halved.

Q4. What does coder 0 vs 1 mean?
A. 0 = Latin-1, read one byte per character; 1 = UTF-16, read two bytes per character.

Q5. Why can String cache its hash?
A. Because it is immutable, the content and therefore the hash never changes.

String Constructors

Besides the literal form, `new String(...)` has many overloads. All of them build a fresh object in the heap:

Argument	Result
`new String()` or `new String("")`	an empty string (length 0; not null)
`new String("Hello")`	a copy of the given String
`new String(char[] arr)`	the whole character array converted to a String
`new String(char[] arr, int offset, int count)`	`count` characters starting at index `offset`
`new String(byte[] b)` / `(b, offset, count)`	bytes interpreted as character codes (97, 98, 99 → "abc")
`new String(StringBuilder sb)` / `(StringBuffer sb)`	the builder's current content as a String

```
char[] arr = {'J', 'a', 'v', 'a', '!'};
String whole = new String(arr);              // Java!
String part  = new String(arr, 0, 4);        // Java    (offset 0, count 4)

arr[0] = 'L';                                // changing the array afterwards ...
System.out.println(whole);                   // Java!   ... does NOT affect the String (it copied the data)

byte[] codes = {97, 98, 99};
System.out.println(new String(codes));       // abc
System.out.println(new String(codes, 0, 2)); // ab
```

In the three-argument constructors the third value is a **count** (how many characters), not an end index. This differs from `substring(begin, end)`, where the end index is exclusive.

The String Methods, by Purpose

The class has dozens of methods and **none of them needs memorising**. Know which kinds of operation exist; the IDE lists the rest when you type `s.`, and several methods have overloads. Every method that appears to "modify" a String actually returns a new one. The examples below use `String s = "Programming";`

Length and emptiness

Method	Result
`s.length()`	**11** — a method for Strings (arrays use the field `arr.length`, no brackets)
`s.isEmpty()`	`true` only if length is 0
`s.isBlank()`	(Java 11) `true` if empty *or only white space*

For `" "` (five spaces): length is 5, `isEmpty()` is false (there are characters), `isBlank()` is true (they are all white space).

Character access

```
s.charAt(2)          // 'o'     character at index 2 (indexes start at 0)
s.toCharArray()      // char[] {'P','r','o', ...}   the whole String as a character array
```

Comparison

- `equals(other)` compares the text (String overrides `Object.equals`); `equalsIgnoreCase(other)` ignores upper/lower case.
- `compareTo(other)` compares **lexicographically** — dictionary order — and returns an `int`: negative if this String comes first, 0 if equal, positive if it comes later. The size is the difference of the first differing characters' codes.

```
"abc".compareTo("abd")    // -1   'c' (99) is 1 less than 'd' (100)
"abc".compareTo("abe")    // -2
"abc".compareTo("abc")    //  0
"abd".compareTo("abc")    //  1
```

Searching

```
s.contains("gram")        // true     takes a CharSequence (an interface that String implements)
s.indexOf('g')            // 3        first occurrence (-1 if absent); also accepts a String
s.lastIndexOf('g')        // 10       last occurrence
s.startsWith("Pro")       // true
s.endsWith("ing")         // true
```

`indexOf` has several overloads (a char, a String, and a starting position for each); a char argument is accepted as an `int` code, which is how characters are stored internally.

Extraction and transformation

```
s.substring(1, 4)         // "rog"        begin is INCLUSIVE, end is EXCLUSIVE  ->  indexes 1,2,3
s.substring(3)            // "gramming"   from index 3 to the end
s.toUpperCase()           // "PROGRAMMING"
s.toLowerCase()           // "programming"
"  hi  ".trim()           // "hi"         removes leading and trailing spaces only, never inner ones
"  hi  ".strip()          // "hi"         same idea, but understands Unicode white space (Java 11)
"ab".repeat(3)            // "ababab"
s.replace('m', 'n')       // "Progranning"      char for char
s.replace("gram", "GRAM") // "ProGRAMming"      substring for substring
```

`substring(1, 4)` uses the same mathematical idea as interval notation: `[1, 4)` — the start index is included, the end index is excluded. Use `strip()` instead of `trim()` when the text may contain non-ASCII white space; for ordinary text `trim()` is fine. `replace` substitutes *every* occurrence of a literal text, while `replaceAll(regex, replacement)` takes a regular expression pattern, e.g. `"a1b22c".replaceAll("[0-9]+", "#")` gives `"a#b#c"`.

split and join

`split(delimiter)` breaks one String into a `String[]` at each delimiter; `String.join(delimiter, ...)` (a **static** method) does the opposite. They are used constantly with comma-separated data and for building keys such as `"a-b-c"`.

```
String names = "Ali,Sara,Omar";
String[] arr = names.split(",");              // ["Ali", "Sara", "Omar"]
for (String n : arr) System.out.println(n);

System.out.println(String.join("-", "a", "b", "c"));   // a-b-c
```

Conversion

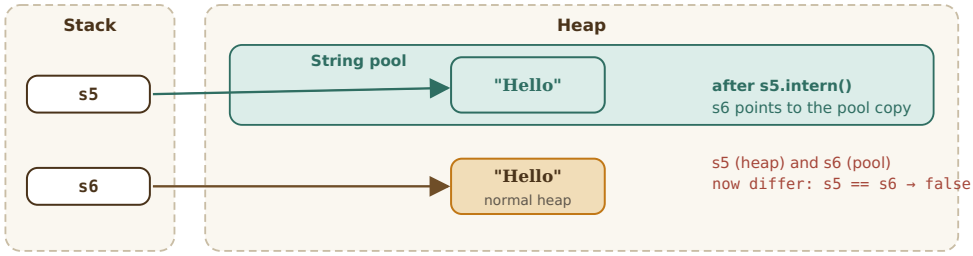
```
String.valueOf(10)        // "10"          static: any primitive -> String (like Integer.valueOf, in reverse)
"Aa".getBytes()           // byte[] {65, 97}   the character codes of the String
s.toCharArray()           // char[]            (seen above)
```

intern() — move a heap String into the pool

Suppose `s5 = new String("Hello")` (heap) and `s6 = s5`, so `s5 == s6` is true. Calling `s5.intern()` asks the JVM for the pool's copy of that text (adding it if absent). Assigning that result makes the variable point into the pool; heap and pool references then differ. It is used to push many duplicate heap Strings into the shared pool and save memory.

```
String s5 = new String("Hello");   // heap object
String s6 = s5;
System.out.println(s5 == s6);       // true

s6 = s5.intern();                   // s6 now points to the POOL copy
System.out.println(s5 == s6);       // false   (heap vs pool)
System.out.println(s6 == "Hello");  // true
```



intern() returns the pooled instance of a String.

String.format()

Building a sentence with many `+` operators is hard to read. `String.format` (static) takes a template with placeholders such as `%s` and fills them in order from the following arguments:

```
String name = "Sara";  int age = 28;

String a = "Hello " + name + ", your age is " + age;                    // hard to read
String b = String.format("Hello %s, your age is %s", name, age);        // clear
// both: Hello Sara, your age is 28
```

Active Recall — Part 3

Q1. length() of a String vs length of an array?

A. String has a method length(); an array has the field length (no brackets).

Q2. isEmpty() vs isBlank() for " "?

A. isEmpty() = false (it has characters), isBlank() = true (only white space). isBlank came in Java 11.

Q3. What does compareTo return for "abc" vs "abe"?

A. -2: lexicographic order, difference of 'c' and 'e'; negative means the first String comes earlier.

Q4. substring(1, 4) of "Programming"?

A. "rog": begin index inclusive, end index exclusive.

Q5. trim() vs strip()? replace vs replaceAll?

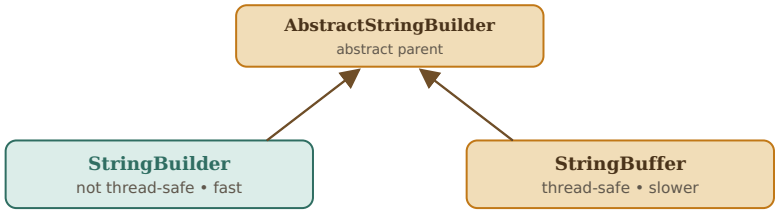
A. strip() is Unicode-aware, trim() handles plain ASCII spaces; replace uses literal text, replaceAll uses a regex.

Q6. split vs join? What does intern() do?

A. split cuts a String into a String[]; static join glues pieces with a delimiter. intern() returns the pooled copy of a String.

StringBuilder and StringBuffer

Both classes hold text that can be **changed in place**, so repeated edits do not create new objects. They differ only in thread safety:



Both extend AbstractStringBuilder; all their methods match, only the locking differs.

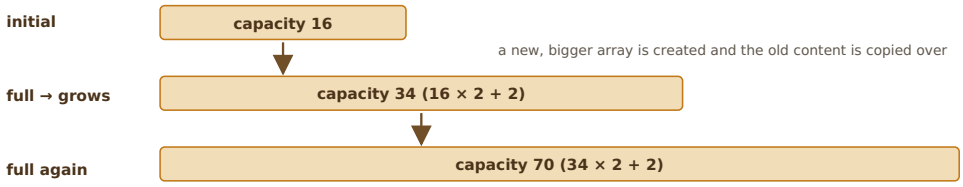
Both live in `java.lang` and are built like this inside:

```
class StringBuilder extends AbstractStringBuilder {
    byte[] value;      // the buffer holding the characters
    int    count;      // how many positions are actually used
    // (an internal 'coder' also exists, exactly as in String, but it is not exposed)
}
```

The `value` array is deliberately **larger than the text**, leaving free room at the end. When you `append`, characters go into that spare room — no new object. Two numbers describe the situation: **count**/ `length()` is how many positions are used, **capacity()** is the total size of the array including the empty part.

Capacity and growth

- `new StringBuilder()` → capacity **16**.
- `new StringBuilder(50)` → capacity **50** (you choose it).
- `new StringBuilder("Java")` → capacity **16 + 4 = 20** (16 spare plus the length of the text).
- When the array fills up, a new array of size **old × 2 + 2** is created and the content is copied: 16 → 34 → 70 and so on. This happens automatically; you just keep calling `append`.



The buffer doubles (plus 2) whenever it fills up — a dynamic array.

The methods

```
StringBuilder sb = new StringBuilder();
sb.append("Hamza");               // Hamza
sb.append(" Ali");                // Hamza Ali
sb.insert(2, 'O');                // HaOmza Ali      insert at index 2, the rest shifts right
sb.delete(0, 2);                  // Omza Ali        removes indexes 0 and 1  (end exclusive)
sb.deleteCharAt(0);               // mza Ali         removes one character
sb.replace(1, 3, "XY");           // mXY Ali         replaces indexes 1..2 with "XY"
sb.reverse();                     // ilA YXm         whole text reversed
sb.charAt(0);                     // character at an index
sb.setCharAt(0, 'Z');             // overwrite the character at an index
sb.length();                      // positions in use
sb.capacity();                    // size of the internal array
```

Method	What it does
`append(x)`	add at the end
`insert(i, x)`	add in the middle at index *i*
`delete(start, end)` / `deleteCharAt(i)`	remove a range / one character
`replace(start, end, str)`	replace a range with new text
`reverse()`	reverse the whole content
`charAt(i)` / `setCharAt(i, c)`	read / overwrite a character
`ensureCapacity(n)`	make sure capacity is at least *n* (jumps straight to it, e.g. 34 → 100)
`trimToSize()`	shrink capacity down to the current length, freeing unused space
`toString()`	get a normal (immutable) String copy of the content

```
StringBuilder sb = new StringBuilder();      // capacity 16
sb.append("Programming");                    // 11 used
sb.append("Programming");                    // 22 used > 16  -> array grew
System.out.println(sb.length());             // 22
System.out.println(sb.capacity());           // 34
sb.trimToSize();                             // work finished, drop the spare room
System.out.println(sb.capacity());           // 22

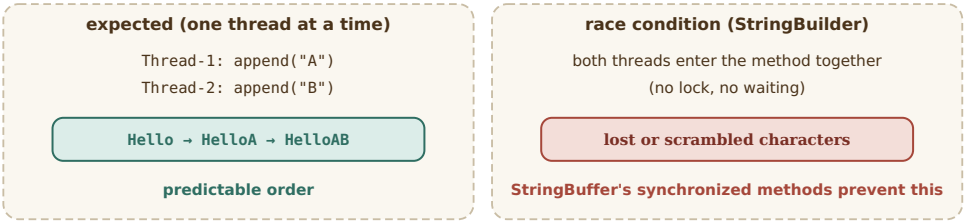
String result = sb.toString();               // back to a String
```

`System.out.println(sb)` also works because `println` calls `toString()` on whatever object it receives.

Interview point: StringBuilder and StringBuffer do **not** override `equals()`. Two builders holding identical text are *not* equal, because `Object.equals()` compares references: `new StringBuilder("a").equals(new StringBuilder("a"))` is **false**. Also, they are not supersets of String: only the manipulation methods exist on them, not every String method.

StringBuffer: the thread-safe twin

`StringBuffer` has exactly the same constructors and methods as `StringBuilder`. The one difference: all its methods are **synchronized** (thread-safe). Imagine two threads appending to the same shared builder at once; without protection both can enter the method together and the result is scrambled — a **race condition**. StringBuffer's locking lets only one thread in at a time. (Threads are covered in depth later.)



Without locking, two threads can interleave and corrupt the text.

	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread-safe?	Yes (cannot change at all)	No	Yes (synchronized)
Speed for edits	Slow (new object each time)	Fastest	Slower (locking overhead)
Typical use	Fixed text	Building text (about 90% of cases)	Rare; shared across threads

In real projects **StringBuilder** is used almost always: its speed matters, a plain String is already safe for sharing because it never changes, and modern code controls thread access explicitly with locks rather than relying on every method of StringBuffer being synchronized.

Active Recall — Part 4

Q1. Why use StringBuilder instead of += in a loop?
A. It edits one buffer in place; += creates a new String every iteration.

Q2. What are the default capacity and the growth formula?
A. 16; when full the new size is old × 2 + 2 (16 → 34 → 70).

Q3. length() vs capacity()? What do ensureCapacity and trimToSize do?
A. length = used positions, capacity = array size. ensureCapacity raises the minimum; trimToSize shrinks capacity to the length.

Q4. Do two StringBuilders with equal text satisfy equals()?
A. No — equals() is not overridden, so it compares references.

Q5. StringBuilder vs StringBuffer, and which is used in practice?
A. Same API; StringBuffer is synchronized (thread-safe) but slower. StringBuilder is used in about 90% of real code.